

# A Guide to Unix Using Linux

## Fourth Edition

### *Chapter 4*

#### *UNIX/Linux File Processing*

# Objectives

- Explain UNIX and Linux file processing
- Use basic file manipulation commands to create, delete, copy, and move files and directories
- Employ commands to combine, cut, paste, rearrange, and sort information in files
- Create a script file

# Objectives (continued)

- Use the join command to link files using a common field
- Use the awk command to create a professional-looking report

# UNIX and Linux File Processing

- Files are treated as nothing more than character sequences
- Concept offers a lot of flexibility
  - You can directly access each character
  - You can perform a range of editing tasks

# Reviewing UNIX/Linux File Types

- **Regular files (“-”)**
  - Text files
    - Contain printable ASCII characters
    - Sometimes called regular/ordinary/ASCII files
    - Examples: documents, source code, etc.
  - Binary files
    - Contain nonprintable characters
    - Example: machine language code
- Directories (“d”)
- Special files
  - Character special files (“c”), block special files (“b”)

# Understanding File Structures

- Different ways to structure files
  - **Flat ASCII file:** created, manipulated, and used to store data (e.g., letters, product reports)
  - Record structure:
    - **Variable-length record:** typically separated by a delimiter

```
219:432:4567:Harrison:Joel:M:4540:Accountant:09-12-1985
```

- **Fixed-length record:** each field has a specified length

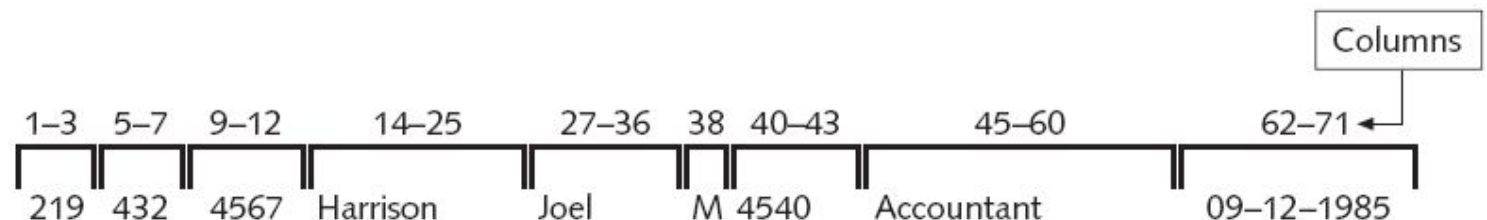


Figure 4-1 Fixed-length record

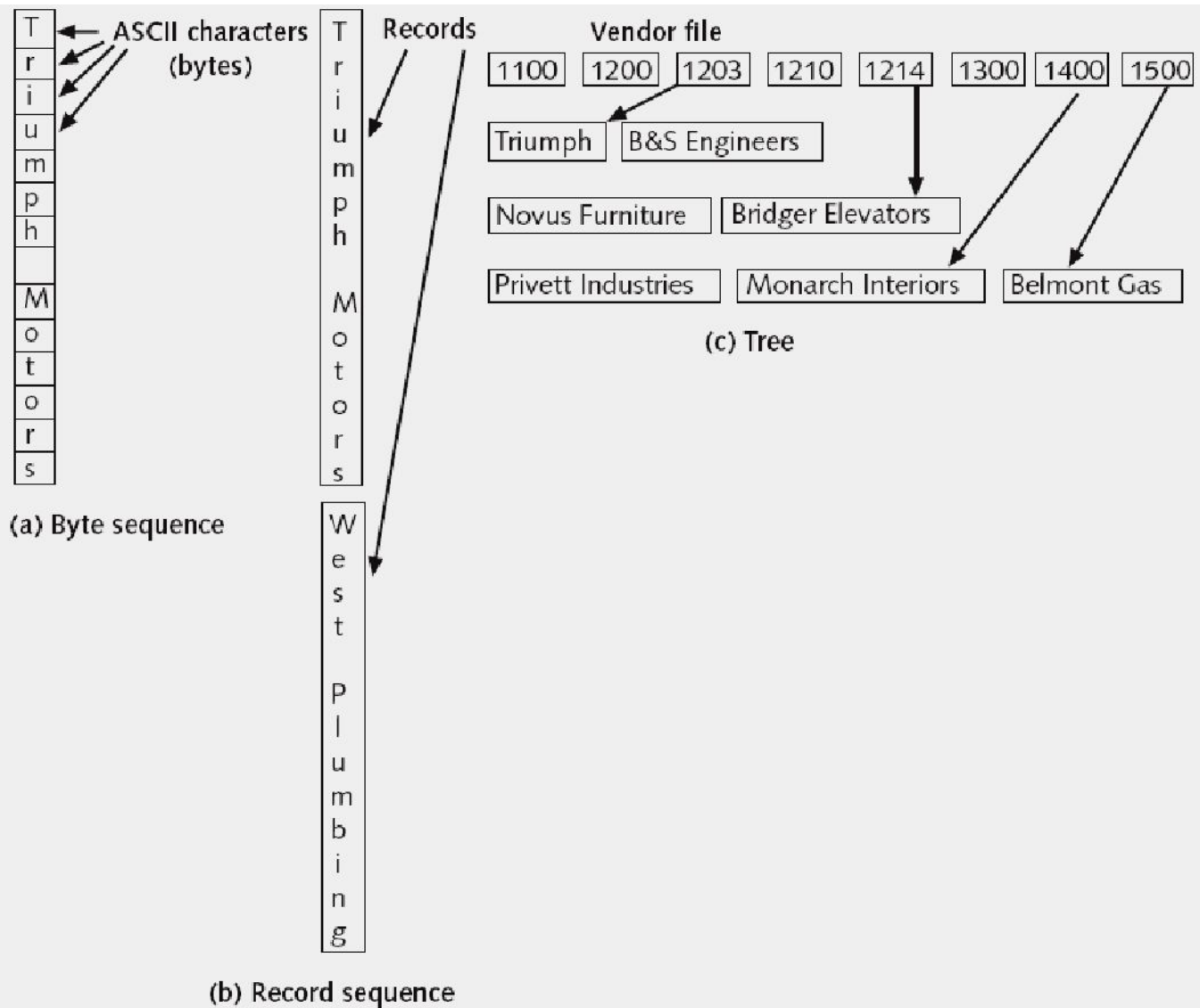


Figure 4-2 Three kinds of text files

# Processing Files

- **stdin:** standard input
  - Keyboard
- **stdout:** standard output
  - Monitor or console
- **stderr:** standard error
  - Screen
- Can be redirected

# Using Input and Error Redirection

- Use `>` and `>>` to redirect output
  - Example: `ls > homedir.list`
- Use `<` and `<<` to redirect input
  - Example: `vi testfile < commands`
- Use `2>` to redirect commands or program error messages
  - Example: `ls Fellowese 2> errors`

# Manipulating Files

- Some ways to manipulate files:
  - Create files
  - Delete files
  - Remove directories
  - Copy files
  - Move files
  - Find files
  - Combine files
  - Combine files through pasting
  - Extract fields in files through cutting
  - Sort files

# Creating Files

- Two simple ways to create files:
  - > `accountsf`
  - `touch accountsf2`
- Primary purpose of *touch* is to change a file's time stamp and date stamp

---

*Syntax* **touch** [-options] *filename(s)*

---

## *Dissection*

- Intended to change the time stamp on a file, but can also be used to create an empty file
  - Useful options include:
    - a updates the access time only
    - m updates the last time the file was modified
    - c prevents the *touch* command from creating the file, if it does not already exist
-

# Deleting Files

- Delete a file using the *rm* (remove) command
  - Example: *rm test\**

---

*Syntax* **rm** [-options] *filename* or *directoryname*

---

## *Dissection*

- Used to delete files or directories
  - Useful options include:
    - i displays a warning prompt before deleting the file (or directory)
    - r when deleting a directory, recursively deletes its files and subdirectories (to delete a directory that is empty or that contains entries, use the -r option with *rm*)
-

# Removing Directories

- Use *rm* or *rmdir* to remove an empty directory

*Syntax* **rmdir** [-options] *directoryname*

## *Dissection*

- Used to delete directories
- A directory must be empty to delete it with the *rmdir* command.

- Use *rm -r* to remove a non-empty directory

# Copying Files

- Use *cp* for copying files

*Syntax* **cp** [-options] *source destination*

## *Dissection*

- Used to copy files or directories
- Useful options include:
  - i provides a warning before *cp* writes over an existing file with the same name
  - s creates a symbolic link or name at the destination rather than a physical file (a symbolic name is a pointer to the original file, which you learn about in Chapter 6)
  - u prevents *cp* from copying over an existing file if the existing file is newer than the source file

- **Examples:**

```
cp class_of_88 duplicates/classmates
cp project1 project2 project3 duplicates
cp designs/* duplicates
```

# Moving Files

- To move a file, use *mv* (move) along with the source file name and destination name
  - As insurance, a file is copied before it is moved
  - Moving and renaming a file are the same operation

---

*Syntax* **mv** [-options] *source destination*

---

## *Dissection*

- Used to move and to rename files
  - Useful options include:
    - i displays a warning prompt before overwriting a file with the same name
    - u overwrites a destination file with the same name, if the source file is newer than the one in the destination
-

# Finding Files

- To search for files with a specified name, use *find*

---

*Syntax* **find** [*pathname* ] [**-name** *filename* ]

---

## *Dissection*

- Used to locate files in a directory and in subdirectories
- Useful options include:

*pathname* is the path name of the directory you want to search. The *find* command searches recursively; that is, it starts in the named directory and searches down through all files and subdirectories under the directory specified by *pathname*.

*-name* indicates that you are searching for files with a specific *filename*. You can use wildcard characters in the file name. For example, you can use *phone\** to search for all file names that begin with “phone.”

*-iname* works like *-name*, but ignores case. For example, if you search for *phone\** as the search name, you’ll find all files that begin with “phone,” “Phone,” “PHONE,” or any combination of upper and lowercase letters.

*-mmin n* displays files that have been changed within the last *n* minutes.

*-mtime n* displays files that have been changed within the last *n* days.

*-size n* displays files of size *n*, where the default measure for *n* is in 512-byte blocks (you can also use *nc*, *nk*, *nM*, or *nG* for bytes kilobytes, megabytes, or gigabytes, such as *find -size 2M* to find files that are 2 megabytes). For other search conditions you can use with *find*, refer to Appendix B, “Syntax Guide to UNIX/Linux Commands.”

---

# Combining Files

- You can use *cat* to combine files
- For example:

```
cat janes_research marks_research > total_research
```

# Combining Files with the paste Command

---

*Syntax* **paste** [-options] *source files* [*> destination file* ]

---

## *Dissection*

- Combines the contents of one or more files to output to the screen or to another file
- By default, the pasted results appear in columns separated by tabs
- Useful options include:
  - d enables you to specify a different separator (other than a tab) between columns
  - s causes files to be pasted one after the other instead of in parallel

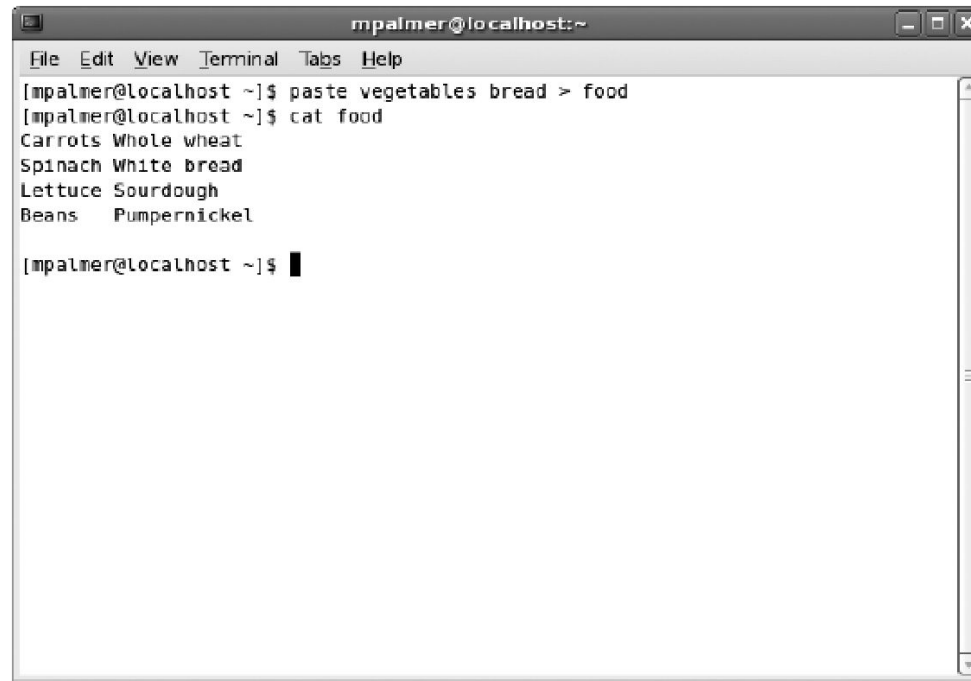
- 
- For example, two files (vegetables and bread):

|         |
|---------|
| Carrots |
| Spinach |
| Lettuce |
| Beans   |

|              |
|--------------|
| Whole wheat  |
| White bread  |
| Sourdough    |
| Pumpernickel |

- Can be pasted using *paste vegetables bread > food*

# Combining Files with the paste Command (continued)

A terminal window titled 'mpalmer@localhost:~' with a menu bar (File, Edit, View, Terminal, Tabs, Help). The terminal shows the following commands and output:

```
[mpalmer@localhost ~]$ paste vegetables bread > food
[mpalmer@localhost ~]$ cat food
Carrots Whole wheat
Spinach White bread
Lettuce Sourdough
Beans Pumpernickel
[mpalmer@localhost ~]$
```

Figure 4-3 Using the *paste* command to merge files

- **Another example:**

```
paste -d', ' vegetables bread > food
```

# Extracting Fields Using the cut Command

---

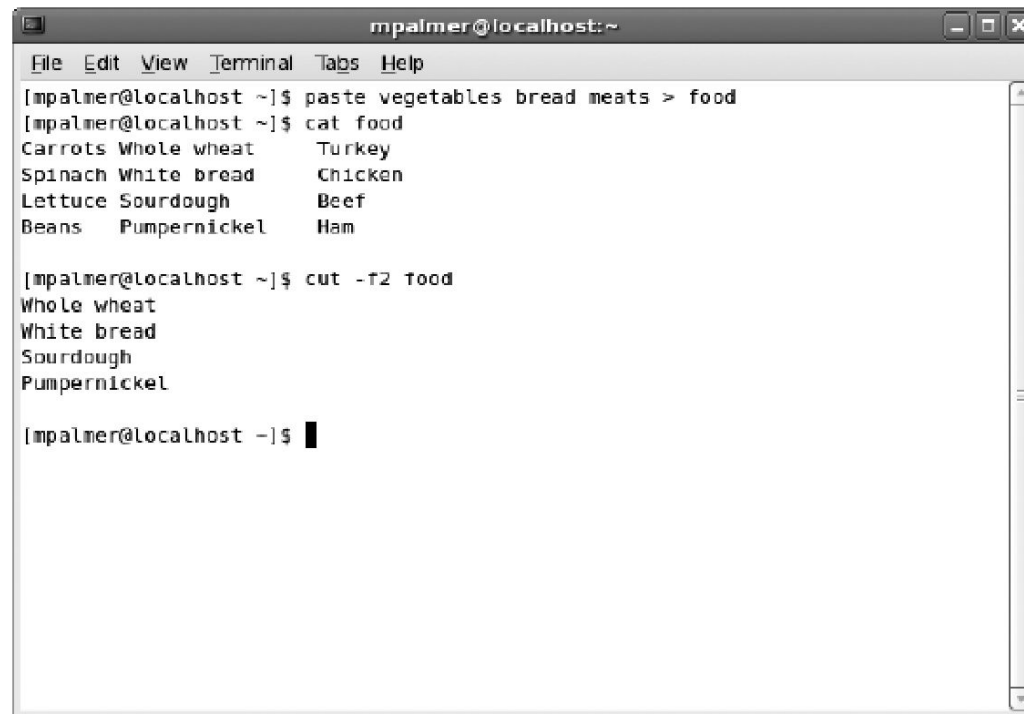
*Syntax* **cut** [-f *list*] [-d **char**] [*file1 file2 . . .*] or **cut** [-c *list*] [*file1 file2 . . .*]

---

## *Dissection*

- Removes specific columns or fields from a file
- Useful options include:
  - f specifies that you are referring to fields
  - list* is a comma-separated list or a hyphen-separated range of integers that specifies the field. For example, -f 1 indicates field 1, -f 1,14 indicates fields 1 and 14, and -f 1-14 indicates fields 1 through 14.
  - d indicates that a specific character separates the fields
  - char* is the character used as the field separator (delimiter), for example, a comma. The default field delimiter is the tab character.
  - file1, file2* are the files from which you want to cut columns or fields
  - c references character positions. For example, -c 1 specifies the first character and -c 1,14 specifies characters 1 and 14.

# Extracting Fields Using the cut Command (continued)



```
mpalmer@localhost:~  
File Edit View Terminal Tabs Help  
[mpalmer@localhost ~]$ paste vegetables bread meats > food  
[mpalmer@localhost ~]$ cat food  
Carrots Whole wheat Turkey  
Spinach White bread Chicken  
Lettuce Sourdough Beef  
Beans Pumpernickel Ham  
  
[mpalmer@localhost ~]$ cut -f2 food  
Whole wheat  
White bread  
Sourdough  
Pumpernickel  
  
[mpalmer@localhost ~]$
```

Figure 4-4 Using the *cut* command

# Sorting Files

*Syntax* **sort** [-options] [*filename* ]

## *Dissection*

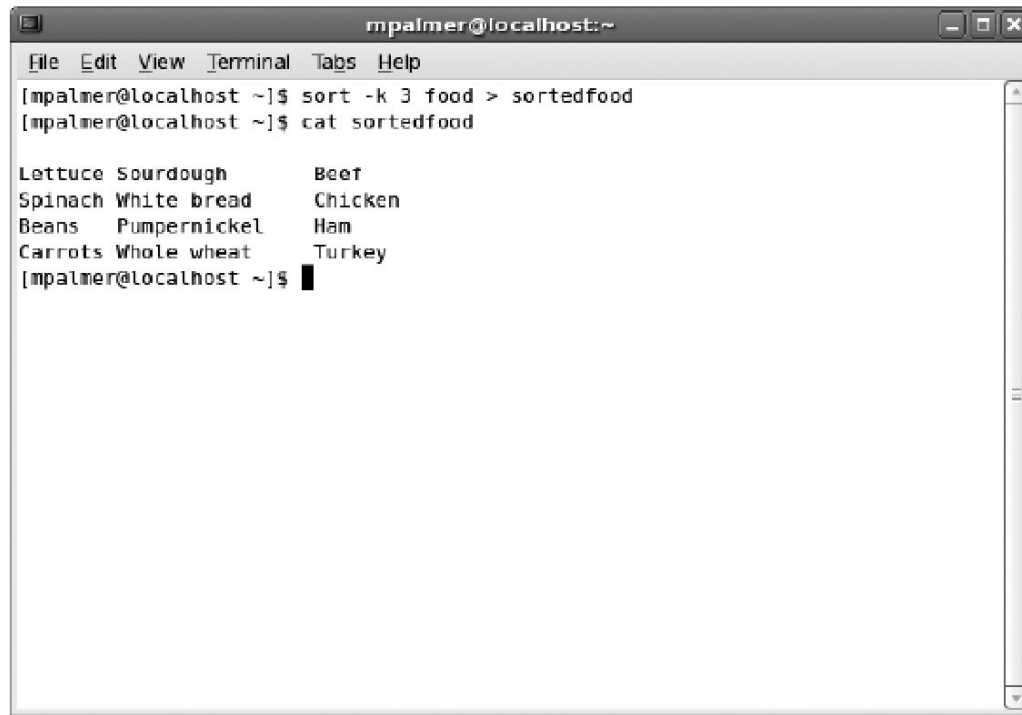
- Sorts the contents of files by individual lines
- Useful options include:
  - k n* sorts on the key field specified by *n*
  - t* indicates that a specified character separates the fields
  - m* merges input files that have been previously sorted (does not perform a sort)
  - o* redirects output to the specified file
  - d* sorts in alphanumeric or dictionary order
  - g* sorts by numeric (general) order
  - r* sorts in reverse order

- **Examples:**

```
sort file1 > file2
```

```
sort -k 3 food > sortedfood
```

# Sorting Files (continued)



A terminal window titled 'mpalmer@localhost:~' with a menu bar (File, Edit, View, Terminal, Tabs, Help). The terminal shows the following commands and output:

```
[mpalmer@localhost ~]$ sort -k 3 food > sortedfood
[mpalmer@localhost ~]$ cat sortedfood
```

|         |              |         |
|---------|--------------|---------|
| Lettuce | Sourdough    | Beer    |
| Spinach | White bread  | Chicken |
| Beans   | Pumpernickel | Ham     |
| Carrots | Whole wheat  | Turkey  |

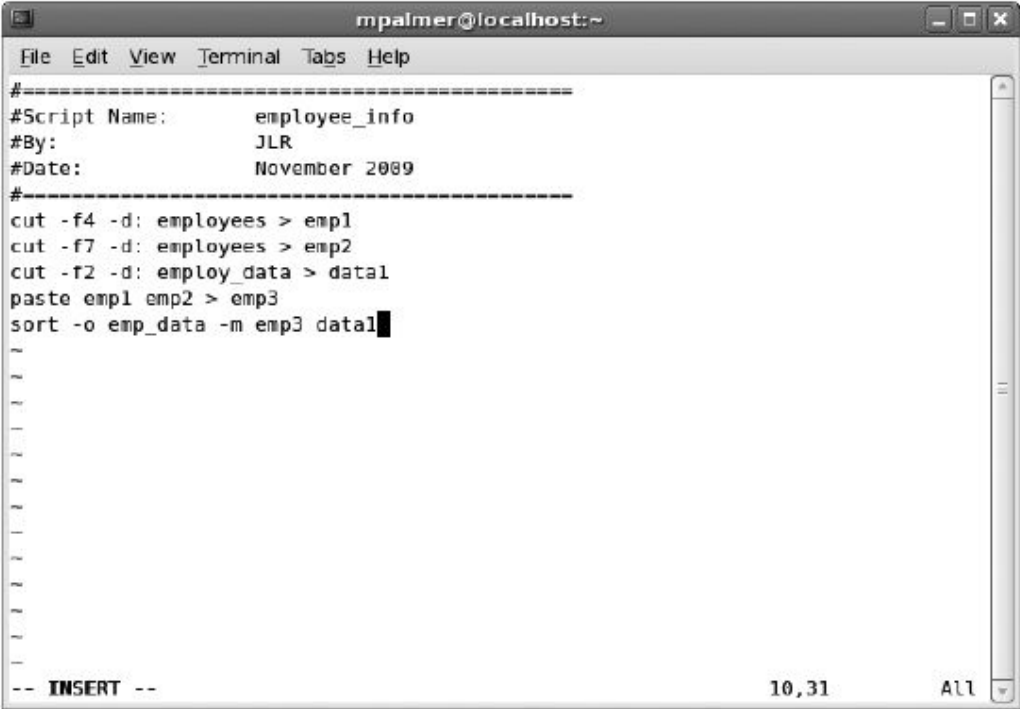
[mpalmer@localhost ~]\$

Figure 4-5 Results of sorting on the third field in the food file

# Creating Script Files

- To automate tasks, MS-DOS and Windows users create batch files
  - Commands are executed when file is run
- UNIX/Linux users do the same:
  - **Shell script** contains command-line entries
- Steps:
  - Create script using a text editor (e.g., vi, Emacs)
  - Make file executable (use *chmod*)
  - Execute (e.g., *./myscript*)

# Creating Script Files (continued)



```
mpalmer@localhost:~
File Edit View Terminal Tabs Help
=====
#Script Name:      employee_info
#By:               JLR
#Date:            November 2009
#=====
cut -f4 -d: employees > emp1
cut -f7 -d: employees > emp2
cut -f2 -d: employ_data > data1
paste emp1 emp2 > emp3
sort -o emp_data -m emp3 data1
-- INSERT --
10,31 All
```

Figure 4-6 Sample script file

# Using the join Command on Two Files

- Use *join* to associate lines in two files on the basis of a common field in them
  - Example:

```
Brown:82:53,000
Anders:110:32,000
Caplan:174:41,000
Crow:95:36,000
```

```
Brown:LaVerne:F:Accounting Department:444-7508: . . .
Anders:Carol:M:Sales Department:444-2130: . . .
Caplan:Jason:R:Payroll Department:444-5609: . . .
Crow:Lorretta:L:Shipping Department:444-8901: . . .
```

Files above can be joined to obtain:

```
Brown:LaVerne:Accounting Department:53,000
Anders:Carol:Sales Department:32,000
Caplan:Jason:Payroll Department:41,000
Crow:Lorretta:Shipping Department:36,000
```

# Using the join Command on Two Files (continued)

*Syntax* **join** [-options] *file1 file2*

## *Dissection*

- Used to associate information in two different files on the basis of a common field or key in those files
- *file1, file2* are two input files that must be sorted on the join field—the field you want to use to join the files. The join field is also called a key. You must sort the files before you can join them. When you issue the *join* command, UNIX/Linux compare the two fields. Each output line contains the common field followed by a line from *file1* and then a line from *file2*. You can modify output using the options described next. If records with duplicate keys are in the same file, UNIX/Linux join on all of them. You can create output records for unpairable lines, for example, to append data from one file to another without losing records.
- Useful options include:
  - 1 *fieldnum* specifies the common field in *file1* on which to join
  - 2 *fieldnum* specifies the common field in *file 2* on which to join
  - o specifies a list of fields to output. The list contains blank-separated field specifiers in the form *m.n*, where *m* is the file number and *n* is the position of the field in the file. Thus, -o 1.2 means “output the second field in the first file.”
  - t specifies the field separator character. By default this is a blank, tab, or new line character. Multiple blanks and tabs count as one field separator.
  - a *filenum* produces a line for each unpairable line in the file *filenum*. (In this case, *filenum* is a 1 for *file1* or a 2 for *file2*.)
  - e *str* replaces the empty fields for the unpairable line in the string specified by *str*. The string is usually a code or message to indicate the condition, for example, -e “No Vendor Record.”

# A Brief Introduction to the Awk Program

- Awk: pattern-scanning and processing language
  - Helps to produce reports that look professional
  - Inventors: A. Aho, P. Weinberger, and B. Kernighan

---

*Syntax* **awk** [- **Fsep** ] [*pattern* {*action*} ..] *filenames*

---

## *Dissection*

- *awk* checks to see if the input records in the specified files satisfy the *pattern* and, if they do, *awk* executes the *action* associated with it. If no pattern is specified, the action affects every input record.
- *-F*: means the field separator is a colon

## – Example:

```
awk 'BEGIN { print "This is an awk print line." }'
```

# A Brief Introduction to the Awk Program (continued)

- Some of the tasks you can do with *awk* include:
  - Manipulate fields and records in a data file
  - Use variables
  - Use arithmetic, string, and logical operators
  - Execute commands from a shell script
  - Use classic programming logic, such as loops
  - Process/organize data into well-formatted reports

- Another example:

```
awk -F: '{printf "%s\t %s\n", $1, $2}' datafile
```

# Summary

- UNIX/Linux support regular files, directories, character special files, and block special files
  - Three kinds of regular files: unstructured ASCII characters, records, and trees
  - Often, flat ASCII data files contain records and fields
  - Standard devices: stdin, stdout, and stderr
- *touch* updates a file's time/date stamp
  - Also used to create empty files
- *rmdir* removes an empty directory
  - Use *rm -r* to remove non-empty directories

# Summary (continued)

- *cut* extracts specific columns or fields from a file
- *paste*: combines two or more files
- *sort*: sorts a file's contents
- Create shell scripts to automate tasks
- *join*: extracts information from two files sharing a common field
- Awk is a pattern-scanning and processing language
  - Creates a formatted report with a professional look

# Command Summary

| Command     | Purpose                                                                    | Options Covered in This Chapter                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>awk</b>  | Starts the <i>awk</i> program to format output                             | <b>-F</b> identifies the field separator.<br><b>-f</b> indicates code is coming from a disk file, not the keyboard.                                                                                                                                                                                                                                                                                                                                                      |
| <b>cat</b>  | Views the contents of a file, creates a file, merges the contents of files |                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>cp</b>   | Copies one or more files                                                   | <b>-i</b> provides a warning before <i>cp</i> writes over an existing file with the same name.<br><b>-s</b> creates a symbolic link or name at the destination rather than a physical file.<br><b>-u</b> prevents <i>cp</i> from copying over an existing file, if the existing file is newer than the source file.                                                                                                                                                      |
| <b>cut</b>  | Extracts specified columns or fields from a file                           | <b>-c</b> refers to character positions.<br><b>-d</b> indicates that a specified character separates the fields.<br><b>-f</b> refers to fields.                                                                                                                                                                                                                                                                                                                          |
| <b>find</b> | Finds files                                                                | <b>-iname</b> specifies the name of the files you want to locate, but the search is not case sensitive.<br><b>-name</b> specifies the name of the files you want to locate, but the search is case sensitive.<br><b>-mmin</b> <i>n</i> displays files that have been changed within the last <i>n</i> minutes.<br><b>-mtime</b> <i>n</i> displays files that have been changed within the last <i>n</i> days.<br><b>-size</b> <i>n</i> displays files of size <i>n</i> . |

| Command      | Purpose                                                                         | Options Covered in This Chapter                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>join</b>  | Combines files having a common field                                            | <ul style="list-style-type: none"> <li>-a <i>n</i> produces a line for each unpairable line in file <i>n</i>.</li> <li>-e <i>str</i> replaces the empty fields for an unpairable file with the specified string.</li> <li>-1 and -2 with the field number are used to specify common fields when joining.</li> <li>-o outputs a specified list of fields.</li> <li>-t indicates that a specified character separates the fields.</li> </ul> |
| <b>mv</b>    | Moves one or more files                                                         | <ul style="list-style-type: none"> <li>-i displays a warning prompt before overwriting a file with the same name.</li> <li>-u overwrites a destination file with the same name, if the source file is newer than the one in the destination.</li> </ul>                                                                                                                                                                                     |
| <b>paste</b> | Combines fields from two or more files                                          | <ul style="list-style-type: none"> <li>-d enables you to specify a different separator (other than a tab) between columns.</li> <li>-s causes files to be pasted one after the other instead of in parallel.</li> </ul>                                                                                                                                                                                                                     |
| <b>rm</b>    | Removes one or more files                                                       | <ul style="list-style-type: none"> <li>-i specifies that UNIX/Linux should request confirmation of file deletion before removing the files.</li> <li>-r specifies that directories should be recursively removed.</li> </ul>                                                                                                                                                                                                                |
| <b>rmdir</b> | Removes an empty directory                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>sort</b>  | Sorts the file's contents                                                       | <ul style="list-style-type: none"> <li>-k <i>n</i> sorts on the key field specified by <i>n</i>.</li> <li>-t indicates that a specified character separates the fields.</li> <li>-m means to merge files before sorting.</li> <li>-o redirects output to the specified file.</li> <li>-d sorts in alphanumeric or dictionary order.</li> <li>-g sorts by numeric (general) order.</li> <li>-r sorts in reverse order.</li> </ul>            |
| <b>touch</b> | Updates an existing file's time stamp and date stamp or creates empty new files | <ul style="list-style-type: none"> <li>-a specifies that only the access date and time are to be updated.</li> <li>-m specifies that only the modification date and time are to be updated.</li> <li>-c specifies that no files are to be created.</li> </ul>                                                                                                                                                                               |